EXPT NO: 8 DATE: 2/11/2026	TRAIN A SIMPLE AUTOENCODER FOR DIMENSIONALITY REDUCTION AND FEATURE LEARNING

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is an autoencoder?
An autoencoder is an unsupervised neural network that learns to compress input data into a compact representation and reconstruct the original data. It is commonly used for dimensionality reduction, noise removal, anomaly detection, and feature learning.

2. What are the roles of the encoder and decoder in an autoencoder?
The **encoder** compresses the input data into a lower-dimensional representation called the latent representation. The **decoder** reconstructs the original input from this representation. Together, they help the autoencoder learn meaningful features while minimizing the difference between the input and reconstructed output.

3. What is dimensionality reduction?
Dimensionality reduction is the process of reducing the number of features or variables in a dataset while preserving important information. It reduces computational complexity, removes redundant features, and simplifies data analysis. Autoencoders perform nonlinear dimensionality reduction by learning compressed representations.

4. What is meant by latent representation or latent space?
Latent representation is a compressed numerical representation of input data learned by the encoder. Latent space is the space containing these representations. It captures important features of the input in a lower-dimensional form, making data representation and analysis more efficient.

5. Why is reconstruction loss used when training an autoencoder?
Reconstruction loss measures the difference between the original input and the reconstructed output. It helps the autoencoder learn to reproduce the input accurately by updating its network parameters during training. Common loss functions include Mean Squared Error (MSE) and Binary Cross-Entropy (BCE).

AIM
To implement and train a simple autoencoder using a dataset such as MNIST, learn a lower-dimensional representation of the input data, and evaluate the model by reconstructing the input samples and visualizing their latent representations.

SOFTWARE REQUIREMENTS
- Python
- TensorFlow / Keras
- Google Colab or Jupyter Notebook
- GitHub
- NumPy
- Matplotlib
- Pandas (optional)

OBJECTIVES
Students will be able to:
- Understand the architecture and working of autoencoders.
- Differentiate between the encoder, latent representation, and decoder.

- Prepare and preprocess a dataset for autoencoder training.
- Build an autoencoder using appropriate neural network layers.
- Train the model using reconstruction loss.
- Perform dimensionality reduction and extract latent features.
- Visualize original and reconstructed samples.
- Evaluate reconstruction performance using loss values and visual inspection.

PROCEDURE

PART A: DATASET PREPARATION AND PREPROCESSING

Perform the following tasks:

- Load the MNIST handwritten digit dataset using TensorFlow/Keras.
- Explore the dataset and check the number of training and testing images and their dimensions.
- Display sample images from the dataset.
- Convert pixel values to floating-point values and normalize them to the range [0, 1].
- Flatten each 28 × 28 image into a 784-dimensional input vector for a basic dense autoencoder.
- Prepare training and testing data. Use the input images as both input and target output because the autoencoder learns to reconstruct its input.

PART B: IMPLEMENTING THE ENCODER AND DECODER

Perform the following tasks:

- Define the input layer with 784 features.
- Build the encoder using one or more Dense layers to map the input to a smaller latent dimension, such as 32 features.
- Build the decoder using Dense layers to map the latent representation back to 784 output values.
- Use suitable activation functions, such as ReLU for hidden layers and Sigmoid for the final layer.
- Combine the encoder and decoder into a single autoencoder model.
- Display the encoder, decoder, and autoencoder model summaries.

PART C: COMPILING AND TRAINING THE AUTOENCODER

Perform the following tasks:

- Compile the autoencoder using an optimizer such as Adam.
- Select an appropriate reconstruction loss, such as binary cross-entropy or mean squared error.
- Train the model by supplying the input images as both input data and target data.
- Use a suitable batch size and number of epochs. For example, use a batch size of 256 and 10 epochs.
- Validate the model using the test dataset or a validation split.
- Record training and validation loss for each epoch.
- Plot the training and validation loss curves to observe the reconstruction performance.

PART D: DIMENSIONALITY REDUCTION, FEATURE LEARNING AND RECONSTRUCTION

Perform the following tasks:

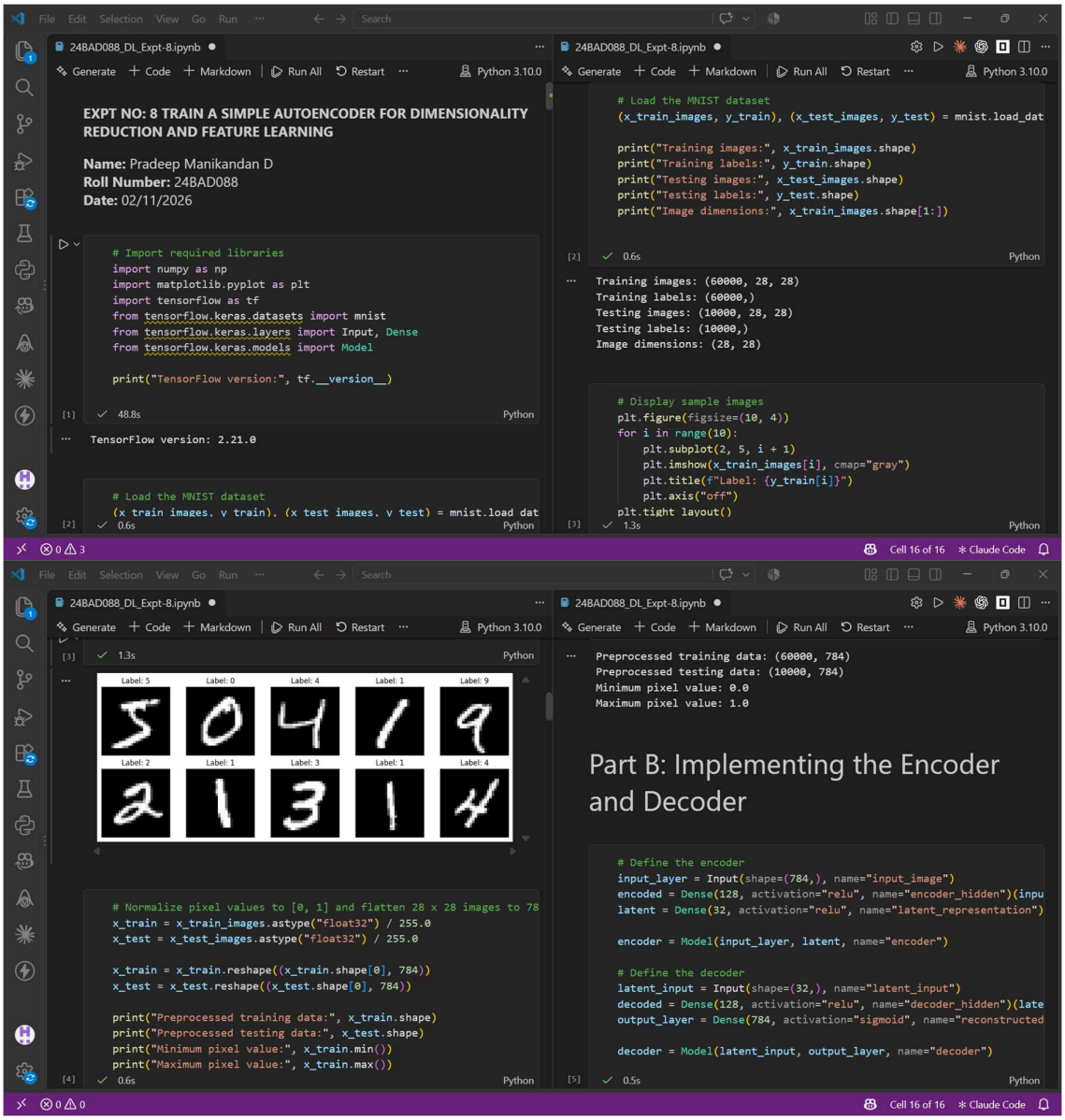
- Use the trained encoder to convert test images into lower-dimensional latent feature vectors.
- Display the original input dimension (784) and latent dimension (for example, 32).
- Pass the latent vectors through the decoder to reconstruct the images.
- Display original images and their corresponding reconstructed images side by side.

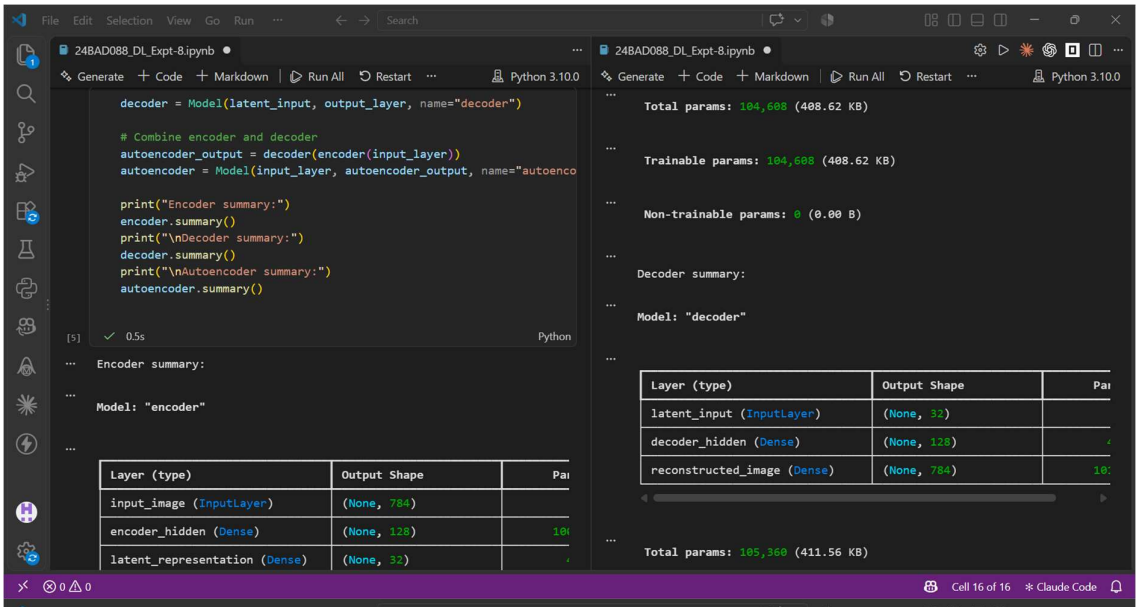
- Compare the original and reconstructed images and comment on the information retained or lost.
- Optionally, use PCA or t-SNE to visualize latent features in two dimensions, if covered in the course.
- Save the trained model or reconstructed images if required.

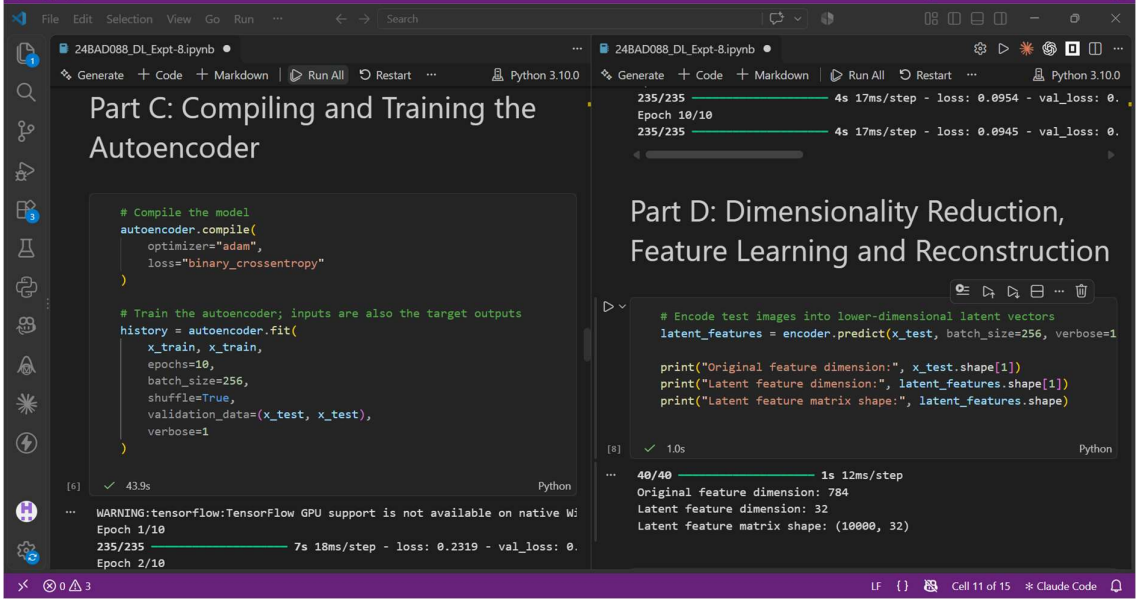
EXPECTED OUTCOME

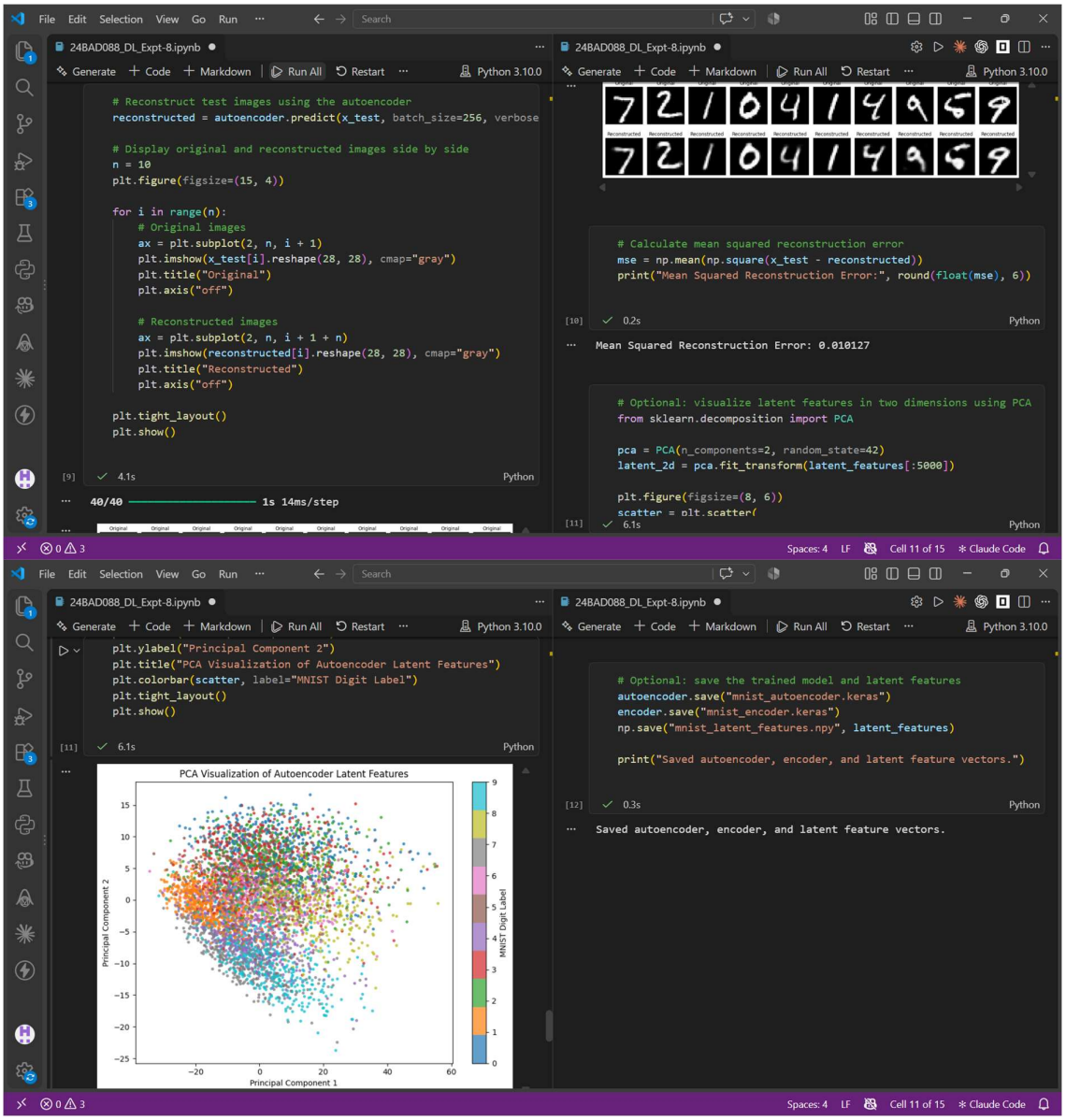
Students will understand the architecture and working of a simple autoencoder and the roles of its encoder, latent representation, and decoder. They will learn to preprocess data, train the model using reconstruction loss, reduce input dimensionality, and extract compact latent features. They will evaluate the autoencoder by examining training and validation loss and comparing original images with reconstructed images.

Screenshots




24BAD088_DL_Expt-8.ipynb
Generate
Code
Markdown
Run All
Restart
Python 3.10.0
decoder = Model(latent_input, output_layer, name="decoder")
Combine encoder and decoder
autoencoder_output = decoder(encoder(input_layer))
autoencoder = Model(input_layer, autoencoder_output, name="autoenco
print("Encoder summary:")
encoder.summary()
print("\nDecoder summary:")
decoder.summary()
print("\nAutoencoder summary:")
autoencoder.summary()
[5] 0.5s Python
Encoder summary:
Model: "encoder"
Layer (type) Output Shape Pa
input_image (InputLayer) (None, 784)
encoder_hidden (Dense) (None, 128) 10
latent_representation (Dense) (None, 32)
Total params: 104,608 (408.62 KB)
Trainable params: 104,608 (408.62 KB)
Non-trainable params: 0 (0.00 B)
Decoder summary:
Model: "decoder"
Layer (type) Output Shape Pa
latent_input (InputLayer) (None, 32)
decoder_hidden (Dense) (None, 128)
reconstructed_image (Dense) (None, 784) 10
Total params: 105,360 (411.56 KB)
Cell 16 of 16
Claude Code


24BAD088_DL_Expt-8.ipynb
Generate
Code
Markdown
Run All
Restart
Python 3.10.0
Part C: Compiling and Training the Autoencoder
Compile the model
autoencoder.compile(
optimizer="adam",
loss="binary_crossentropy"
)
Train the autoencoder; inputs are also the target outputs
history = autoencoder.fit(
x_train, x_train,
epochs=10,
batch_size=256,
shuffle=True,
validation_data=(x_test, x_test),
verbose=1
)
[6] 43.9s Python
WARNING:tensorflow:TensorFlow GPU support is not available on native W
Epoch 1/10
235/235 7s 18ms/step - loss: 0.2319 - val_loss: 0.
Epoch 2/10
235/235 4s 17ms/step - loss: 0.0954 - val_loss: 0.
Epoch 10/10
235/235 4s 17ms/step - loss: 0.0945 - val_loss: 0.
Part D: Dimensionality Reduction, Feature Learning and Reconstruction
Encode test images into lower-dimensional latent vectors
latent_features = encoder.predict(x_test, batch_size=256, verbose=1
print("Original feature dimension:", x_test.shape[1])
print("Latent feature dimension:", latent_features.shape[1])
print("Latent feature matrix shape:", latent_features.shape)
[8] 1.0s Python
40/40 1s 12ms/step
Original feature dimension: 784
Latent feature dimension: 32
Latent feature matrix shape: (10000, 32)
Cell 11 of 15
Claude Code



POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is the main purpose of an autoencoder?

An autoencoder is an unsupervised neural network that learns to compress input data into a compact representation and reconstruct the original data. Its main purpose is to learn important features while reducing unnecessary information. It is used for dimensionality reduction, noise removal, anomaly detection, and feature extraction.

2. How do the encoder and decoder differ?

The **encoder** converts the input data into a lower-dimensional representation by extracting important features. The **decoder** uses this compressed representation to reconstruct the original input as accurately as possible. Both components work together to minimize reconstruction loss during training.

3. What is a latent representation?

A latent representation is a compressed numerical representation of input data produced by the encoder. It captures the essential features of the input while reducing its dimensionality. The

collection of these representations forms the latent space, which can be used for feature learning and data analysis.

4. How does an autoencoder perform dimensionality reduction?
An autoencoder performs dimensionality reduction by compressing high-dimensional input data into a smaller representation using its encoder. This representation preserves important features while reducing the number of dimensions. The decoder reconstructs the original input from the compressed representation, helping the model learn meaningful patterns.

5. Why are the input images also used as target outputs during training?
Input images are used as target outputs because an autoencoder learns to reconstruct its own input rather than predict a separate label. The model compares the reconstructed image with the original image and adjusts its parameters to minimize reconstruction loss. This allows it to learn features without labeled data.

6. What does reconstruction loss measure?
Reconstruction loss measures the difference between the original input and the output reconstructed by the autoencoder. It indicates how accurately the model reproduces the input data. A lower reconstruction loss generally means better reconstruction. Common loss functions include Mean Squared Error (MSE) and Binary Cross-Entropy (BCE).

7. What may happen if the latent dimension is too small?
If the latent dimension is too small, the autoencoder may not have enough capacity to preserve all the important information from the input. This can lead to information loss, higher reconstruction error, and poor-quality outputs. Therefore, an appropriate latent dimension is important for effective compression.

8. Mention one real-world application of autoencoders.
Anomaly detection is an important real-world application of autoencoders. The model learns patterns from normal data and reconstructs similar samples accurately. Unusual samples may produce higher reconstruction errors, helping identify fraudulent transactions, defective products, and abnormal network activity.

SUGGESTIONS FOR DATASETS

- MNIST Handwritten Digits – https://www.tensorflow.org/datasets/catalog/mnist
- Fashion-MNIST – https://www.tensorflow.org/datasets/catalog/fashion_mnist
- CIFAR-10 – https://www.tensorflow.org/datasets/catalog/cifar10 (requires adapting the model for image inputs)

GitHub: DL expt 8

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
	Faculty Signature	